API de Controle de Manutenção Industrial

Documentação dos Endpoints — NP1 Prova Subjetiva

Desenvolvimento Web Back-End · Engenharia de Software · UniATENEU

Outubro de 2026

1. Visão geral

API RESTful para gerenciar o setor de manutenção de uma indústria: equipamentos, ordens de serviço, defeitos, peças substituídas e controle de manutenção preventiva. Todas as requisições e respostas usam JSON.

Item	Valor
URL base	`http://localhost:3000`
Linguagem / framework	Node.js + Express
Banco de dados	SQLite nativo do Node.js (módulo `node:sqlite`), arquivo `manutencao.db` criado automaticamente
Requisito	Node.js 22.13 ou mais novo
Formato	JSON (enviar o header `Content-Type: application/json`)
Datas	Formato `AAAA-MM-DD` (ex.: 2026-10-08)

2. Como executar

```
npm install      (instala as dependências - só na primeira vez)
npm start        (liga a API em http://localhost:3000)
npm run zerar    (apaga o banco de dados - com o servidor desligado)
```

Testes no Postman: importe o arquivo `docs/Colecao-Postman.json` (Import → arquivo). Com o banco vazio, abra a coleção e clique em **Run** para executar todas as requisições em ordem. Cada requisição tem um teste automático que confere o código de resposta.

3. Estrutura do projeto

```
manutencao-industrial/
├── server.js              inicia o Express, registra as rotas e trata erros gerais
├── database.js            conexão com o SQLite e criação das tabelas
├── utils.js               funções de apoio (datas, validação de ID e de campos)
├── routes/
│   ├── equipamentos.js    CRUD de equipamentos
│   ├── preventivas.js     controle de manutenção preventiva
│   ├── ordensServico.js   CRUD de ordens de serviço
│   ├── defeitos.js        CRUD de defeitos
│   └── pecas.js           CRUD de peças substituídas
└── docs/                  documentação e coleção do Postman
```

4. Modelo de dados

Tabela	Campos
equipamentos	id, nome, modelo, fabricante, data_instalacao, status (Ativo | Em manutenção | Inativo), periodicidade_dias, ultima_manutencao, proxima_manutencao
ordens_servico	id, equipamento_id → equipamentos, tipo (Preventiva | Corretiva), data_abertura, data_conclusao, responsavel, status (Aberta | Em andamento | Finalizada)
defeitos	id, equipamento_id → equipamentos, descricao, severidade (Baixo | Médio | Alto | Crítico), data_registro
pecas	id, ordem_servico_id → ordens_servico, nome, codigo, quantidade, custo_unitario, data_substituicao

As setas (→) são chaves estrangeiras: ligam a ordem de serviço e o defeito ao equipamento, e a peça à ordem de serviço.

5. Códigos de resposta

Código	Quando acontece
200 OK	Consulta, alteração ou exclusão feita com sucesso
201 Created	Cadastro feito com sucesso
400 Bad Request	ID inválido, campos obrigatórios ausentes, valor inválido, JSON mal escrito ou ordem de serviço já finalizada
404 Not Found	Registro (equipamento, ordem, defeito, peça) ou rota não encontrada
409 Conflict	Exclusão bloqueada porque o registro tem outros dados ligados a ele
500 Internal Server Error	Erro inesperado no servidor

Toda resposta de erro tem o formato `{ "erro": "mensagem" }`.

6. Resumo dos endpoints

Método	URL	Descrição
GET	`/api/equipamentos`	Listar equipamentos
GET	`/api/equipamentos/:id`	Buscar equipamento por ID
GET	`/api/equipamentos/em-manutencao`	Equipamentos em manutenção
POST	`/api/equipamentos`	Cadastrar equipamento
PUT	`/api/equipamentos/:id`	Atualizar equipamento
DELETE	`/api/equipamentos/:id`	Excluir equipamento

GET	/api/preventivas	Controle de manutenção preventiva
GET	/api/preventivas/vencidas	Alerta de preventivas vencidas
PUT	/api/preventivas/:id	Registrar periodicidade e última manutenção
GET	/api/ordens-servico	Listar ordens de serviço
GET	/api/ordens-servico/:id	Buscar ordem de serviço por ID
GET	/api/ordens-servico/:id/custo-total	Custo total das peças da ordem
POST	/api/ordens-servico	Abrir ordem de serviço
PUT	/api/ordens-servico/:id	Atualizar / finalizar ordem de serviço
DELETE	/api/ordens-servico/:id	Excluir ordem de serviço
GET	/api/defeitos	Listar defeitos (mais graves primeiro)
GET	/api/defeitos/:id	Buscar defeito por ID
POST	/api/defeitos	Registrar defeito
PUT	/api/defeitos/:id	Atualizar defeito
DELETE	/api/defeitos/:id	Excluir defeito
GET	/api/pecas	Listar peças substituídas
GET	/api/pecas/:id	Buscar peça por ID
GET	/api/pecas/historico/:codigo	Histórico de substituições de uma peça
POST	/api/pecas	Registrar peça substituída
PUT	/api/pecas/:id	Atualizar peça
DELETE	/api/pecas/:id	Excluir peça

7. Endpoints detalhados

As respostas abaixo são reais, geradas executando as requisições na ordem da coleção do Postman. Listas longas aparecem resumidas (2 primeiros itens). Datas preenchidas automaticamente usam o dia em que a requisição é feita.

Equipamentos (CRUD)

Cadastrar equipamento

POST `http://localhost:3000/api/equipamentos`

Cadastra um equipamento. Campos obrigatórios: nome, modelo, fabricante, data_instalacao (AAAA-MM-DD). status é opcional (padrão "Ativo"; aceitos: Ativo, Em manutenção, Inativo).

Body (JSON)

```
{
  "nome": "Torno CNC",
  "modelo": "TX-500",
  "fabricante": "Romi",
  "data_instalacao": "2022-03-15",
  "status": "Ativo"
}
```

Resposta esperada — 201 Created

```
{
  "id": 1,
  "nome": "Torno CNC",
  "modelo": "TX-500",
  "fabricante": "Romi",
  "data_instalacao": "2022-03-15",
  "status": "Ativo",
  "periodicidade_dias": null,
  "ultima_manutencao": null,
  "proxima_manutencao": null
}
```

Listar equipamentos

GET `http://localhost:3000/api/equipamentos`

Lista todos os equipamentos cadastrados.

Body

Não tem body.

Resposta esperada — 200 OK


```
[
  {
    "id": 1,
    "nome": "Torno CNC",
    "modelo": "TX-500",
    "fabricante": "Romi",
    "data_instalacao": "2022-03-15",
    "status": "Ativo",
    "periodicidade_dias": null,
    "ultima_manutencao": null,
    "proxima_manutencao": null
  },
  {
    "id": 2,
    "nome": "Prensa Hidráulica",
    "modelo": "PH-200",
    "fabricante": "Schuler",
    "data_instalacao": "2021-07-10",
    "status": "Ativo",
    "periodicidade_dias": null,
    "ultima_manutencao": null,
    "proxima_manutencao": null
  }
]
```


Buscar equipamento por ID

GET `http://localhost:3000/api/equipamentos/1`

Busca um equipamento pelo ID.

Body

Não tem body.

Resposta esperada — 200 OK


```
{
  "id": 1,
  "nome": "Torno CNC",
  "modelo": "TX-500",
  "fabricante": "Romi",
  "data_instalacao": "2022-03-15",
  "status": "Ativo",
  "periodicidade_dias": null,
  "ultima_manutencao": null,
  "proxima_manutencao": null
}
```


Atualizar equipamento (Problema 1)

PUT `http://localhost:3000/api/equipamentos/1`

Corrige os dados de um equipamento. Só os campos enviados são alterados.

Body (JSON)

```
{
  "modelo": "TX-550",
  "fabricante": "Romi S.A."
}
```

Resposta esperada — 200 OK

```
{
  "id": 1,
  "nome": "Torno CNC",
  "modelo": "TX-550",
  "fabricante": "Romi S.A.",
  "data_instalacao": "2022-03-15",
  "status": "Ativo",
  "periodicidade_dias": null,
  "ultima_manutencao": null,
  "proxima_manutencao": null
}
```

Excluir equipamento sem vínculos

DELETE `http://localhost:3000/api/equipamentos/3`

Exclui um equipamento. Só é permitido se ele não tiver ordens de serviço nem defeitos vinculados.

Body

Não tem body.

Resposta esperada — 200 OK

```
{
  "mensagem": "Equipamento excluído com sucesso"
}
```

Manutenção Preventiva

Registrar periodicidade - Torno CNC

PUT `http://localhost:3000/api/preventivas/1`

Registra a periodicidade (em dias) e a última manutenção do equipamento (o ID da URL é do equipamento). A próxima manutenção é calculada: última + periodicidade.

Body (JSON)

```
{
  "periodicidade_dias": 30,
  "ultima_manutencao": "2026-08-01"
}
```

Resposta esperada — 200 OK

```
{
  "equipamento_id": 1,
  "nome": "Torno CNC",
  "periodicidade_dias": 30,
  "ultima_manutencao": "2026-08-01",
  "proxima_manutencao": "2026-08-31",
  "situacao": "VENCIDA"
}
```

Listar controle de preventivas

GET `http://localhost:3000/api/preventivas`

Lista a periodicidade, a última e a próxima manutenção de cada equipamento, com a situação (Em dia ou VENCIDA).

Body

Não tem body.

Resposta esperada — 200 OK

```
[
  {
    "id": 1,
    "nome": "Torno CNC",
    "periodicidade_dias": 30,
    "ultima_manutencao": "2026-08-01",
    "proxima_manutencao": "2026-08-31",
    "situacao": "VENCIDA"
  },
  {
    "id": 2,
    "nome": "Prensa Hidráulica",
    "periodicidade_dias": 90,
    "ultima_manutencao": "2026-09-15",
    "proxima_manutencao": "2026-12-14",
    "situacao": "Em dia"
  }
]
```

Alerta de preventivas vencidas (Problema 3)

GET `http://localhost:3000/api/preventivas/vencidas`

Lista só os equipamentos com a manutenção preventiva vencida (próxima manutenção antes de hoje), com mensagem de alerta.

Body

Não tem body.

Resposta esperada — 200 OK

```
{
  "total_vencidas": 1,
  "equipamentos": [
    {
      "id": 1,
      "nome": "Torno CNC",
      "periodicidade_dias": 30,
      "ultima_manutencao": "2026-08-01",
      "proxima_manutencao": "2026-08-31",
      "alerta": "ATENÇÃO: manutenção preventiva vencida desde 2026-08-31"
    }
  ]
}
```

Ordens de Serviço (CRUD)

Abrir ordem de serviço corretiva

POST `http://localhost:3000/api/ordens-servico`

Abre uma ordem de serviço. Campos obrigatórios: equipamento_id, tipo (Preventiva ou Corretiva), responsavel. data_abertura é opcional (padrão: hoje). A ordem começa com status "Aberta" e o equipamento passa para "Em manutenção".

Body (JSON)

```
{
  "equipamento_id": 1,
  "tipo": "Corretiva",
  "responsavel": "Carlos Silva"
}
```

Resposta esperada — 201 Created

```
{
  "id": 1,
  "equipamento_id": 1,
  "tipo": "Corretiva",
  "data_abertura": "2026-10-08",
  "data_conclusao": null,
  "responsavel": "Carlos Silva",
  "status": "Aberta",
  "equipamento": "Torno CNC"
}
```

Listar ordens de serviço

GET `http://localhost:3000/api/ordens-servico`

Lista todas as ordens de serviço com o nome do equipamento.

Body

Não tem body.

Resposta esperada — 200 OK


```
[
  {
    "id": 1,
    "equipamento_id": 1,
    "tipo": "Corretiva",
    "data_abertura": "2026-10-08",
    "data_conclusao": null,
    "responsavel": "Carlos Silva",
    "status": "Aberta",
    "equipamento": "Torno CNC"
  },
  {
    "id": 2,
    "equipamento_id": 2,
    "tipo": "Preventiva",
    "data_abertura": "2026-10-08",
    "data_conclusao": null,
    "responsavel": "Ana Souza",
    "status": "Aberta",
    "equipamento": "Prensa Hidráulica"
  }
]
```


Buscar ordem de serviço por ID

GET `http://localhost:3000/api/ordens-servico/1`

Busca uma ordem de serviço pelo ID.

Body

Não tem body.

Resposta esperada — 200 OK


```
{
  "id": 1,
  "equipamento_id": 1,
  "tipo": "Corretiva",
  "data_abertura": "2026-10-08",
  "data_conclusao": null,
  "responsavel": "Carlos Silva",
  "status": "Aberta",
  "equipamento": "Torno CNC"
}
```


Atualizar ordem de serviço

PUT `http://localhost:3000/api/ordens-servico/1`

Altera uma ordem de serviço (status aceitos: Aberta, Em andamento, Finalizada). Só os campos enviados são alterados. Ordem já finalizada não pode ser alterada.

Body (JSON)

```
{
  "status": "Em andamento"
}
```

Resposta esperada — 200 OK


```
{
  "id": 1,
  "equipamento_id": 1,
  "tipo": "Corretiva",
  "data_abertura": "2026-10-08",
  "data_conclusao": null,
  "responsavel": "Carlos Silva",
  "status": "Em andamento",
  "equipamento": "Torno CNC"
}
```


Equipamentos em manutenção (Problema 7)

GET `http://localhost:3000/api/equipamentos/em-manutencao`

Lista os equipamentos com status "Em manutenção".

Body

Não tem body.

Resposta esperada — 200 OK


```
[
  {
    "id": 1,
    "nome": "Torno CNC",
    "modelo": "TX-550",
    "fabricante": "Romi S.A.",
    "data_instalacao": "2022-03-15",
    "status": "Em manutenção",
    "periodicidade_dias": 30,
    "ultima_manutencao": "2026-08-01",
    "proxima_manutencao": "2026-08-31"
  },
  {
    "id": 2,
    "nome": "Prensa Hidráulica",
    "modelo": "PH-200",
    "fabricante": "Schuler",
    "data_instalacao": "2021-07-10",
    "status": "Em manutenção",
    "periodicidade_dias": 90,
    "ultima_manutencao": "2026-09-15",
    "proxima_manutencao": "2026-12-14"
  }
]
```

Peças Substituídas (CRUD)

Registrar peça substituída

POST `http://localhost:3000/api/pecas`

Registra uma peça trocada em uma ordem de serviço. Campos obrigatórios: ordem_servico_id, nome, codigo, quantidade, custo_unitario. data_substituicao é opcional (padrão: hoje).

Body (JSON)

```
{
  "ordem_servico_id": 1,
  "nome": "Rolamento 6205",
  "codigo": "ROL-6205",
  "quantidade": 2,
  "custo_unitario": 35.5
}
```

Resposta esperada — 201 Created

```
{
  "id": 1,
  "ordem_servico_id": 1,
  "nome": "Rolamento 6205",
  "codigo": "ROL-6205",
  "quantidade": 2,
  "custo_unitario": 35.5,
  "data_substituicao": "2026-10-08"
}
```

Listar peças

GET `http://localhost:3000/api/pecas`

Lista todas as peças substituídas.

Body

Não tem body.

Resposta esperada — 200 OK

```
[
  {
    "id": 1,
    "ordem_servico_id": 1,
    "nome": "Rolamento 6205",
    "codigo": "ROL-6205",
    "quantidade": 2,
    "custo_unitario": 35.5,
    "data_substituicao": "2026-10-08"
  },
  {
    "id": 2,
    "ordem_servico_id": 1,
    "nome": "Correia em V A42",
    "codigo": "COR-A42",
    "quantidade": 1,
    "custo_unitario": 89.9,
    "data_substituicao": "2026-10-08"
  }
]
```

Buscar peça por ID

GET `http://localhost:3000/api/pecas/1`

Busca uma peça pelo ID.

Body

Não tem body.

Resposta esperada — 200 OK

```
{
  "id": 1,
  "ordem_servico_id": 1,
  "nome": "Rolamento 6205",
  "codigo": "ROL-6205",
  "quantidade": 2,
  "custo_unitario": 35.5,
  "data_substituicao": "2026-10-08"
}
```

Atualizar peça

PUT `http://localhost:3000/api/pecas/1`

Altera uma peça. Só os campos enviados são alterados. Não é permitido se a ordem de serviço estiver finalizada.

Body (JSON)

```
{
  "quantidade": 3
}
```

Resposta esperada — 200 OK

```
{
  "id": 1,
  "ordem_servico_id": 1,
  "nome": "Rolamento 6205",
  "codigo": "ROL-6205",
  "quantidade": 3,
  "custo_unitario": 35.5,
  "data_substituicao": "2026-10-08"
}
```

Excluir peça

DELETE `http://localhost:3000/api/pecas/4`

Exclui uma peça. Não é permitido se a ordem de serviço estiver finalizada.

Body

Não tem body.

Resposta esperada — 200 OK

```
{
  "mensagem": "Peça excluída com sucesso"
}
```

Custo total da ordem de serviço (Problema 8)

GET `http://localhost:3000/api/ordens-servico/1/custo-total`

Mostra as peças da ordem de serviço, o subtotal de cada uma (quantidade x custo unitário) e o custo total.

Body

Não tem body.

Resposta esperada — 200 OK


```
{
  "ordem_servico_id": 1,
  "pecas": [
    {
      "nome": "Rolamento 6205",
      "codigo": "ROL-6205",
      "quantidade": 3,
      "custo_unitario": 35.5,
      "subtotal": 106.5
    },
    {
      "nome": "Correia em V A42",
      "codigo": "COR-A42",
      "quantidade": 1,
      "custo_unitario": 89.9,
      "subtotal": 89.9
    }
  ],
  "custo_total": 196.4
}
```


Histórico de uma peça (Problema 4)

GET `http://localhost:3000/api/pecas/historico/ROL-6205?ano=2026`

Mostra todas as vezes que a peça (pelo código) foi substituída, com data, ordem de serviço e equipamento. O filtro ?ano= é opcional.

Body

Não tem body.

Resposta esperada — 200 OK


```
{
  "codigo": "ROL-6205",
  "vezes_substituida": 2,
  "quantidade_total": 7,
  "historico": [
    {
      "id": 1,
      "nome": "Rolamento 6205",
      "codigo": "ROL-6205",
      "quantidade": 3,
      "custo_unitario": 35.5,
      "data_substituicao": "2026-10-08",
      "ordem_servico_id": 1,
      "equipamento": "Torno CNC"
    },
    {
      "id": 3,
      "nome": "Rolamento 6205",
      "codigo": "ROL-6205",
      "quantidade": 4,
      "custo_unitario": 35.5,
      "data_substituicao": "2026-10-08",
      "ordem_servico_id": 2,
      "equipamento": "Prensa Hidráulica"
    }
  ]
}
```


Defeitos (CRUD)

Registrar defeito

POST `http://localhost:3000/api/defeitos`

Registra um defeito. Campos obrigatórios: equipamento_id, descricao, severidade (Baixo, Médio, Alto, Crítico). data_registro é opcional (padrão: hoje).

Body (JSON)

```
{
  "equipamento_id": 1,
  "descricao": "Ruído anormal no eixo principal",
  "severidade": "Médio"
}
```

Resposta esperada — 201 Created

```
{
  "id": 1,
  "equipamento_id": 1,
  "descricao": "Ruído anormal no eixo principal",
  "severidade": "Médio",
  "data_registro": "2026-10-08",
  "equipamento": "Torno CNC",
  "prioridade": "Normal"
}
```

Registrar defeito crítico (Problema 5)

POST `http://localhost:3000/api/defeitos`

Um defeito "Crítico" volta com prioridade URGENTE e uma mensagem de alerta.

Body (JSON)

```
{
  "equipamento_id": 2,
  "descricao": "Vazamento de óleo no cilindro hidráulico",
  "severidade": "Crítico"
}
```

Resposta esperada — 201 Created

```
{
  "id": 2,
  "equipamento_id": 2,
  "descricao": "Vazamento de óleo no cilindro hidráulico",
  "severidade": "Crítico",
  "data_registro": "2026-10-08",
  "equipamento": "Prensa Hidráulica",
  "prioridade": "URGENTE",
  "alerta": "DEFEITO CRÍTICO: atendimento imediato!"
}
```

Listar defeitos (mais graves primeiro)

GET `http://localhost:3000/api/defeitos`

Lista os defeitos ordenados por gravidade: Crítico, Alto, Médio, Baixo.

Body

Não tem body.

Resposta esperada — 200 OK


```
[
  {
    "id": 2,
    "equipamento_id": 2,
    "descricao": "Vazamento de óleo no cilindro hidráulico",
    "severidade": "Crítico",
    "data_registro": "2026-10-08",
    "equipamento": "Prensa Hidráulica",
    "prioridade": "URGENTE",
    "alerta": "DEFEITO CRÍTICO: atendimento imediato!"
  },
  {
    "id": 1,
    "equipamento_id": 1,
    "descricao": "Ruído anormal no eixo principal",
    "severidade": "Médio",
    "data_registro": "2026-10-08",
    "equipamento": "Torno CNC",
    "prioridade": "Normal"
  }
]
```


Buscar defeito por ID

GET `http://localhost:3000/api/defeitos/1`

Busca um defeito pelo ID.

Body

Não tem body.

Resposta esperada — 200 OK


```
{
  "id": 1,
  "equipamento_id": 1,
  "descricao": "Ruído anormal no eixo principal",
  "severidade": "Médio",
  "data_registro": "2026-10-08",
  "equipamento": "Torno CNC",
  "prioridade": "Normal"
}
```


Atualizar defeito

PUT `http://localhost:3000/api/defeitos/1`

Altera um defeito. Só os campos enviados são alterados.

Body (JSON)

```
{
  "severidade": "Alto"
}
```

Resposta esperada — 200 OK

```
{
  "id": 1,
  "equipamento_id": 1,
  "descricao": "Ruído anormal no eixo principal",
  "severidade": "Alto",
  "data_registro": "2026-10-08",
  "equipamento": "Torno CNC",
  "prioridade": "Normal"
}
```

Excluir defeito

DELETE `http://localhost:3000/api/defeitos/3`

Exclui um defeito.

Body

Não tem body.

Resposta esperada — 200 OK

```
{
  "mensagem": "Defeito excluído com sucesso"
}
```

Finalizar e excluir Ordem de Serviço

Finalizar ordem de serviço preventiva

PUT `http://localhost:3000/api/ordens-servico/2`

Finaliza a ordem de serviço. A data de conclusão vira a data de hoje, o equipamento volta para "Ativo" (se não tiver outra ordem aberta) e, por ser preventiva, a última e a próxima manutenção do equipamento são atualizadas.

Body (JSON)

```
{
  "status": "Finalizada"
}
```

Resposta esperada — 200 OK

```
{
  "id": 2,
  "equipamento_id": 2,
  "tipo": "Preventiva",
  "data_abertura": "2026-10-08",
  "data_conclusao": "2026-10-08",
  "responsavel": "Ana Souza",
  "status": "Finalizada",
  "equipamento": "Prensa Hidráulica"
}
```

Excluir ordem de serviço

DELETE `http://localhost:3000/api/ordens-servico/3`

Exclui uma ordem de serviço (e as peças dela). Ordem já finalizada não pode ser excluída.

Body

Não tem body.

Resposta esperada — 200 OK

```
{
  "mensagem": "Ordem de serviço excluída com sucesso"
}
```

8. Tratamento de erros

Erro - Equipamento inexistente

GET `http://localhost:3000/api/equipamentos/999`

Busca de um equipamento que não existe.

Body

Não tem body.

Resposta esperada — 404 Not Found

```
{
  "erro": "Equipamento não encontrado"
}
```

Erro - ID inválido

GET `http://localhost:3000/api/equipamentos/abc`

O ID precisa ser um número inteiro positivo.

Body

Não tem body.

Resposta esperada — 400 Bad Request

```
{
  "erro": "ID inválido. O ID deve ser um número inteiro positivo."
}
```

Erro - Campos obrigatórios ausentes

POST `http://localhost:3000/api/equipamentos`

Cadastro sem os campos obrigatórios. A resposta lista quais campos faltam.

Body (JSON)

```
{
  "nome": "Esteira Transportadora"
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "Campos obrigatórios ausentes",
  "campos": [
    "modelo",
    "fabricante",
    "data_instalacao"
  ]
}
```

Erro - Status inválido

POST `http://localhost:3000/api/equipamentos`

Status fora dos valores aceitos.

Body (JSON)

```
{
  "nome": "Esteira Transportadora",
  "modelo": "ET-10",
  "fabricante": "WEG",
  "data_instalacao": "2024-02-01",
  "status": "Quebrado"
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "Status inválido",
  "valores_aceitos": [
    "Ativo",
    "Em manutenção",
    "Inativo"
  ]
}
```

Erro - Ordem de serviço para equipamento inexistente (Problema 2)

POST `http://localhost:3000/api/ordens-servico`

Tentativa de abrir uma ordem de serviço para um equipamento que não existe.

Body (JSON)

```
{
  "equipamento_id": 999,
  "tipo": "Corretiva",
  "responsavel": "Carlos Silva"
}
```

Resposta esperada — 404 Not Found

```
{
  "erro": "Equipamento não encontrado. Não é possível abrir a ordem de serviço."
}
```

Erro - Tipo de manutenção inválido

POST `http://localhost:3000/api/ordens-servico`

O tipo precisa ser Preventiva ou Corretiva.

Body (JSON)

```
{
  "equipamento_id": 1,
  "tipo": "Urgente",
  "responsavel": "Carlos Silva"
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "Tipo de manutenção inválido",
  "valores_aceitos": [
    "Preventiva",
    "Corretiva"
  ]
}
```

Erro - Alterar ordem de serviço já finalizada

PUT `http://localhost:3000/api/ordens-servico/2`

Ordem de serviço finalizada não pode ser alterada.

Body (JSON)

```
{
  "responsavel": "João Pereira"
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "Ordem de serviço já finalizada. Não pode ser alterada."
}
```

Erro - Peça em ordem de serviço finalizada

POST `http://localhost:3000/api/pecas`

Não é possível registrar peça em uma ordem de serviço finalizada.

Body (JSON)

```
{
  "ordem_servico_id": 2,
  "nome": "Vedação do cilindro",
  "codigo": "VED-300",
  "quantidade": 1,
  "custo_unitario": 120
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "Ordem de serviço já finalizada. Não é possível alterar suas peças."
}
```

Erro - Excluir equipamento com ordens de serviço (Problema 6)

DELETE `http://localhost:3000/api/equipamentos/1`

Equipamento com ordens de serviço vinculadas não pode ser excluído (409 Conflict).

Body

Não tem body.

Resposta esperada — 409 Conflict

```
{
  "erro": "Não é possível excluir: o equipamento possui 1 ordem(ns) de serviço vinculada(s).
Altere o status para Inativo."
}
```

Erro - Severidade inválida

POST `http://localhost:3000/api/defeitos`

A severidade precisa ser Baixo, Médio, Alto ou Crítico.

Body (JSON)

```
{
  "equipamento_id": 1,
  "descricao": "Motor aquecendo",
  "severidade": "Grave"
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "Severidade inválida",
  "valores_aceitos": [
    "Baixo",
    "Médio",
    "Alto",
    "Crítico"
  ]
}
```

Erro - Quantidade inválida

POST `http://localhost:3000/api/pecas`

A quantidade precisa ser um número inteiro maior que zero.

Body (JSON)

```
{
  "ordem_servico_id": 1,
  "nome": "Parafuso M8",
  "codigo": "PAR-M8",
  "quantidade": 0,
  "custo_unitario": 1.5
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "quantidade deve ser um número inteiro maior que zero"
}
```

Erro - JSON inválido

POST `http://localhost:3000/api/equipamentos`

Body com JSON mal escrito (vírgula sobrando).

Body (JSON)

```
{
  "nome": "Torno CNC",
}
```

Resposta esperada — 400 Bad Request

```
{
  "erro": "JSON inválido no body da requisição"
}
```

Erro - Rota inexistente

GET `http://localhost:3000/api/maquinas`

Endereço que não existe na API.

Body

Não tem body.

Resposta esperada — 404 Not Found

```
{
  "erro": "Rota não encontrada"
}
```

9. Problemas resolvidos

Problema 1

Uma máquina foi cadastrada incorretamente. Como atualizar suas informações?

Rota: `PUT /api/equipamentos/:id`

Solução: A rota junta os dados atuais com os enviados no body, então só os campos informados mudam. Antes de salvar, valida campos obrigatórios, status e data.

Problema 2

Uma ordem de serviço foi aberta para um equipamento inexistente. Como impedir esse cadastro?

Rota: `POST /api/ordens-servico`

Solução: Antes de inserir, a API procura o equipamento pelo ID. Se não existir, responde 404 e não grava nada. No banco, a coluna equipamento_id também é chave estrangeira (FOREIGN KEY).

Problema 3

Uma manutenção preventiva está vencida. Como o sistema identificará essa situação?

Rota: `GET /api/preventivas/vencidas`

Solução: Cada equipamento guarda a periodicidade e a última manutenção, e a próxima é calculada (última + periodicidade). A rota compara a próxima manutenção com a data de hoje e devolve um alerta para cada equipamento vencido. Quando uma ordem preventiva é finalizada, a última e a próxima manutenção são atualizadas automaticamente.

Problema 4

Uma peça foi substituída diversas vezes ao longo do ano. Como consultar o histórico?

Rota: `GET /api/pecas/historico/:codigo?ano=2026`

Solução: Busca todas as substituições daquele código de peça, em ordem de data, mostrando a ordem de serviço e o equipamento de cada uma, quantas vezes foi trocada e a quantidade total. O filtro por ano é opcional.

Problema 5

Um defeito crítico foi registrado. Como o sistema destacará sua prioridade?

Rota: `POST /api/defeitos e GET /api/defeitos`

Solução: Defeitos com severidade "Crítico" voltam com prioridade "URGENTE" e uma mensagem de alerta. A listagem é ordenada por gravidade, então os críticos aparecem sempre primeiro.

Problema 6

Um usuário tenta excluir um equipamento que possui ordens de serviço vinculadas. Como tratar esse caso?

Rota: `DELETE /api/equipamentos/:id`

Solução: A API conta as ordens de serviço (e os defeitos) do equipamento. Se houver alguma, responde 409 Conflict, não exclui e orienta a mudar o status para "Inativo". Assim o histórico de manutenção não é perdido.

Problema 7

A empresa deseja consultar todos os equipamentos atualmente em manutenção. Como disponibilizar essa consulta?

Rota: `GET /api/equipamentos/em-manutencao`

Solução: A rota lista os equipamentos com status "Em manutenção". O status é atualizado automaticamente: vira "Em manutenção" quando uma ordem de serviço é aberta e volta para "Ativo" quando não sobra nenhuma ordem aberta.

Problema 8

Como calcular o custo total de peças utilizadas em uma ordem de serviço?

Rota: `GET /api/ordens-servico/:id/custo-total`

Solução: O banco soma quantidade x custo_unitario de todas as peças da ordem (SUM no SQL). A resposta mostra cada peça com o subtotal e o custo total.

10. Decisões técnicas

- **Express com um arquivo de rotas por recurso** (`express.Router`): cada parte da API fica separada e fácil de achar.
- **SQLite para persistência**: os dados ficam salvos em um arquivo e não se perdem ao reiniciar. É usado o SQLite que já vem no Node.js (`node:sqlite`), então não precisa instalar servidor de banco nem compilar nada.
- **Chaves estrangeiras** ligando ordens e defeitos ao equipamento, e peças à ordem de serviço, garantindo a integração entre as tabelas.
- **Validação antes de gravar** (ID, campos obrigatórios, valores aceitos, datas, números), com o código HTTP correto e uma mensagem clara em cada erro.
- **PUT que altera só o que foi enviado**: para corrigir um campo errado não é preciso reenviar o registro inteiro.
- **Regras automáticas**: o status do equipamento muda ao abrir e fechar ordens, e a preventiva é recalculada ao finalizar uma ordem preventiva.
- **Exclusão bloqueada (409)** em vez de apagar em cascata, e **ordem finalizada não pode ser alterada**: assim o histórico de manutenção é preservado.